

Context That Pays: Ablation-Calibrated Marginal Context Value Profiles for Budgeted Context Assembly

VENKATA SANGARAJU¹, (Member, IEEE)

¹Independent Researcher, Leander, TX 78641, USA

Corresponding author: Venkata Sangaraju (sangaraju1988@gmail.com)

ABSTRACT LLM agents must fit tool schemas, documents, examples, notes and history into a token budget. Common heuristics (relevance, coverage, compression, fixed priorities) do not measure how much each kind of context changes task success. We introduce marginal context value (MCV) profiles: per model and task family, the expected success drop when a context type is removed, the value of its short variant, and pairwise type interactions. Each is estimated once by counterfactual ablation with bootstrap intervals and a length-neutral control. A CP-SAT compiler selects blocks under the budget, with dependencies and variants, in milliseconds without calling the language model. In a pre-registered, fully local evaluation on Qwen3-4B-Instruct (multi-hop QA over HotpotQA, BFCL tool calling, long-history recall) against six baselines including LLMingua-2 and a coverage selector, the primary hypothesis failed. MCV did not significantly exceed embedding relevance in area under the success–budget curve in any family, and trailed it by 0.053 on history, where its block ranker favored recent over relevant sessions. The efficiency hypothesis, added after development pilots but before any test run, held in two of three families: at an 8k budget MCV sent 34.9% fewer prompt tokens on tool calling and 6.9% fewer on history with non-inferior success; since full contexts are shorter than 8k tokens, this measures pruning rather than budgeted selection. Interaction terms had no effect. The 4B profiles showed that worked examples carried little individual value, and Qwen3-4B and Qwen3-8B type values agreed ($\rho = 0.88$ over 7 pooled pairs); on Qwen3-8B a transferred profile gave the same outcome as a natively measured one in every paired episode, which also indicates low sensitivity of the compiler to the type profile. Code and logs of every model call are released.

INDEX TERMS Context engineering, LLM agents, prompt assembly, token budgets, counterfactual ablation.

I. INTRODUCTION

An LLM agent’s prompt is assembled, not written. Before each decision a harness collects tool schemas, retrieved documents, worked examples, notes and conversation history, and must fit them into a token budget set by latency, cost or the model’s window [1]. What to keep is usually decided by heuristics: a relevance score against the goal, a coverage objective over candidate passages [2], a compressor that deletes tokens [3], or fixed priorities written by an engineer. None of these asks the question a budgeted system actually faces: *how much does this kind of context change task success for this model on this task family?* The answer is not obvious. Models use long contexts unevenly [4], and a block can be relevant to the goal yet add nothing the model needs, or be redundant with another block that is already included.

This paper measures that quantity directly and uses it. A *marginal context value (MCV) profile* records, for a (model, task family) pair, the expected drop in success when every block of a type is removed, the value of the type’s short variant, and the pairwise interaction between types, each with a bootstrap interval. Profiles are estimated once, offline, by counterfactual ablation on development instances, with a length-neutral control that separates content from prompt length. An instance-level

block-value model, trained on leave-one-block-out deltas from the same runs, ranks individual blocks. At inference time a compiler solves a budgeted selection program with dependencies, variants and interaction terms using CP-SAT and returns a certificate; it does not call the language model.

Prior work values context in related ways but for other decisions: attribution methods explain one generation by ablating its sources [5], [6], Shapley-style methods value demonstrations, prompts, data points or skills [7]–[10], and recent context-allocation studies measure the causal effect of context in generative search [11]. Section II positions MCV profiles against these and against budgeted selectors. The distinguishing combination is a reusable, type-level, interval-reported value table measured per model and task family and compiled under a hard token budget.

We evaluate the approach under a pre-registered protocol (hypotheses, margins, sample sizes and the method version fixed before any test run, recorded as a commit in the released repository; it was not deposited with an external registry) on three task families: multi-hop question answering over HotpotQA documents with distractors, tool selection and argument filling over a 30-tool catalog built from BFCL, and recall of the latest value of a user attribute from 24 synthetic sessions. All models run

locally (Qwen3-4B-Instruct for the main study, Qwen3-8B for transfer). We compare against six baselines, including LLMingua-2 and a PACMS-style coverage selector.

The results are mixed, and we report them as such. **The primary hypothesis failed:** MCV-compiled context did not achieve a higher area under the success–budget curve than the best baseline (embedding relevance) in any family. It was 0.053 lower on history, where at the smallest budget the block-value model kept the session containing the answer in 64% of plans, against 94% for relevance ranking. **The efficiency hypothesis,** added after the development pilots but before any test run, **held** in 2 of three families. At an 8k budget the compiler, which leaves budget unused when the remaining blocks add no predicted value, used 34.9% fewer prompt tokens than full context on tool use and 6.9% fewer on history with non-inferior success. On question answering it used 30.9% fewer, but non-inferiority was not shown. Interaction terms had no measurable effect. The profiles themselves were informative. Tool schemas and documents carry large value (0.85 and 0.35), while worked examples carry little value on their own in all three families. History and the profile note substitute for each other (interaction -0.60). The length-neutral control moved no value by more than 0.05, and the type values of the 4B and 8B models were rank-correlated ($\rho = 0.88$).

Contributions.

- *A pre-registered, fully local evaluation* of eight context policies on three task families with measured token counts, paired tests and Holm correction. Its main findings are negative or qualified: measured context value did not beat relevance ranking for selection, and single-block deletions proved too sparse a signal to train a block ranker that improves on embedding similarity (Section VI).
- *MCV profiles as a diagnostic:* a definition and estimator of per-(model, family) type values, short-variant values and pairwise interactions with shrinkage, bootstrap intervals and a length-neutral control, which identified low-value context types and, in subsampling, stabilized with ten to twenty development instances (Sections III, IV and VI).
- *A budget compiler with certificates:* a CP-SAT program over blocks, variants, dependencies and type interactions that enforces the budget in the model’s own tokenizer and reports the solver’s status, objective and bound. The certificate covers optimality with respect to the predicted values only, not task success (Section IV).
- *An open artifact:* the mcv package, data adapters, logs of every model call, and the scripts that produce every number and figure in this paper from those logs.

II. BACKGROUND AND RELATED WORK

A. CONTEXT ENGINEERING AND ASSEMBLY

A survey formalizes context engineering as the discipline of retrieving, processing and managing the information supplied to a language model at inference time [1]. Agentic Context Engineering treats contexts as evolving playbooks that accumulate and refine strategies [12], and DSPy compiles declarative pipelines of language-model calls into optimized prompts [13]; both optimize the *content* of context rather than selecting among existing blocks under a budget. Several 2026 systems assemble context for long-horizon agents. ContextPipe organizes assembly as a five-phase, database-style pipeline with a data-source catalog and a cache-aware optimizer, and notes that maximizing utility under a budget with precedence is a precedence-constrained knapsack that it deliberately does not search [14]. ContextRender selects earlier tool results using observed reuse in an execution-dependency graph together with recency and relevance [15]. Context Assembly as the Controlled Variable frames the choice of template, demonstrations and retrieval as the variable a harness controls [16]; ContextEvo learns a context-management policy from long-horizon trajectories [17]; programmatic context management treats context as an environment the agent manipulates [18]; and Imprompt treats prompts as a programmable control surface [19]. These systems decide what enters context through rules, learned policies or relevance signals. None measures, ahead of time, what each kind of block is worth to task success.

B. SELECTION UNDER A TOKEN BUDGET

PACMS casts assembly-time selection as submodular maximum coverage under a knapsack constraint and solves it with a cost-aware greedy over pluggable selectors [2]. Budgeted skill selection has been formulated as regularized submodular maximization with provable bicriteria guarantees, modelling complementarity through coverage and diminishing returns; its authors name interaction terms learned from execution feedback as future work [20]. In retrieval-augmented generation, budget allocation has been approached by calibrated decisions about how much to retrieve [21], by a reader-context diagnostic for budget-constrained selection [22], by dependency-structured memory selection [23], and by constructing knowledge-base context from historical usage for text-to-SQL [24]. Tool schemas compete with retrieved evidence for the same window, which motivates schema compression [25], and budget-tiered protocols evaluate memory strategies for local agents [26]. Long contexts are also used unevenly by models, which perform worse when relevant information sits in the middle [4].

C. COMPRESSION

LLMLingua-2 distills token-level compression into a small classifier that keeps the tokens a teacher marks as essential [3], and COMI compresses context by a marginal-

information-gain score that rewards relevance and penalizes redundancy [27]. A pre-registered randomized trial of compression in production task orchestration shows that savings depend on how compression changes output length [28], and compressing the persistent instructions of tool-using agents traces reliability frontiers [29]. Compression keeps every block but shortens it; our compiler instead chooses which blocks, and which variant of each, to include.

D. VALUING CONTEXT BY COUNTERFACTUALS

Valuing a unit by what its removal costs has a long history in data valuation, e.g. Data Shapley [9], and has been applied to demonstrations [7] and prompts [8]. For context, ContextCite attributes a generation to context sources with a surrogate model [5] and AttriBoT approximates leave-one-out attribution efficiently [6]; both are post-hoc explanations of one output. Closer to assembly, influence-guided context selection uses leave-one-out influence of retrieved passages in RAG [30]; the Laws of Context Allocation replace relevance proxies with a causal leave-one-out probe of generative reliance [11]; Decision-Aware Memory Cards rank candidate units by their estimated effect on the agent’s next action using judges or surrogates [31]; DRSR learns set-level deletion risk for agent history [32]; McClear clears memories with negative utility via cooperative attribution [33]; and SkillsInjector constructs skill context dynamically [34]. The closest work, SkillsSV, values the internal units of a single skill document with structure-aware Shapley values computed from agent rollouts, accounting for dependencies between units [10]. Kernel surrogate models capture second-order interactions in task attribution [35]. An ablation of persistent context files for coding agents found no measurable effect on correctness [36], a reminder that block value must be measured rather than assumed.

E. POSITIONING

All counterfactual methods above value *one kind* of unit (passages, demonstrations, memories, history, or the parts of one skill), *per instance*, for one model. We do not claim to be first to value context by its removal; influence-guided selection [30] and causal context allocation [11] already select context by leave-one-out effects at assembly time, per instance. Our contribution is a reusable, type-level value model across heterogeneous blocks (goal, instructions, tool schemas, documents, examples, history, notes) with cross-type interaction terms, a test of whether it transfers from a small profiling model to larger ones, and an exact dependency-aware compiler that uses it under a hard budget. The novelty is the reuse of type-level values across instances; in our evaluation this component did not improve selection over relevance ranking.

III. PROBLEM FORMULATION AND THREAT/FAILURE MODEL

A. BLOCKS, BUDGETS AND VALUE

Definition 1 (Typed context block). A block $b = (\tau_b, \{x_b^v\}_v, D_b)$ has a type τ_b from a fixed set $\mathcal{T}$ (goal, instructions, tool schema, skill document, fact, example, history turn, note), one or more textual variants x_b^v (full and optionally short), and a set D_b of blocks it depends on. Its cost $c_m(b, v)$ is the number of tokens of its rendered variant under model m ’s tokenizer.

An instance q of a task family f is a set of blocks $\mathcal{B}_q$ together with a deterministic success check. A selection S assigns to each block either a variant or nothing; it is *admissible* under budget B if every mandatory block (goal and instructions) is included, every included block’s dependencies are included, at most one variant per block is chosen, and $\sum_{b \in S} c_m(b, S(b)) \leq B$. A fixed, documented renderer turns S into one prompt, so success $Y_m(q, S) \in \{0, 1\}$ is a function of the selection and the model’s decoding.

Definition 2 (Marginal context value profile). For model m and family f , the MCV of type τ is $\text{MCV}(\tau) = \mathbb{E}_q[Y_m(q, \mathcal{B}_q) - Y_m(q, \mathcal{B}_q \setminus \tau)]$, the expected loss in success when every block of type τ is removed from the full context. The pairwise interaction of types τ_1, τ_2 is $I(\tau_1, \tau_2) = \mathbb{E}_q[Y(\mathcal{B}) - Y(\mathcal{B} \setminus \tau_1) - Y(\mathcal{B} \setminus \tau_2) + Y(\mathcal{B} \setminus \{\tau_1, \tau_2\})]$, positive when the two types are complements and negative when they are redundant or in conflict. The short-variant value $\text{MCV}_{\text{short}}(\tau)$ is defined like $\text{MCV}(\tau)$ with type τ rendered short instead of removed. A profile is the table of these quantities with confidence intervals.

B. THE ASSEMBLY PROBLEM

Given a profile, an instance and a budget B , the compiler must choose an admissible selection that maximizes predicted value. Type-level values say how much a *kind* of block is worth; instances contain many blocks of one type (dozens of tool schemas or documents), so the compiler also needs instance-level block values $\hat{v}_b$, predicted from cheap features. The resulting problem is a knapsack with dependency constraints, variant choice and pairwise type-presence terms; it is NP-hard in general, and with negative interactions the objective is not submodular, so greedy guarantees for coverage objectives [2], [20] do not apply.

C. WHAT IS AND IS NOT GUARANTEED

Given its inputs, the compiler returns an admissible selection that is optimal for the predicted objective whenever the solver reports optimality, and it fails loudly if the mandatory blocks alone exceed the budget; it never silently truncates. Nothing guarantees that predicted value equals realized success: profiles are estimated from finite ablations on a development split, block values are predicted from features, and transfer to another model is an empirical question. These are exactly the claims Section VI tests.

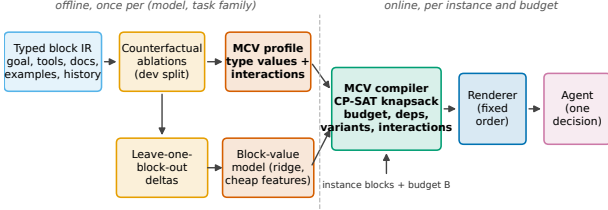


FIGURE 1. MCV profiles and the MCV compiler. Profiling happens once per (model, task family) on development data; compilation happens per instance and per budget without calling the language model (block embeddings are cached).

IV. DESIGN

Figure 1 shows the pipeline: a typed block IR feeds an offline profiler that runs counterfactual ablations on a development split; the estimator turns outcomes into an MCV profile and an instance-level block-value model; at inference time the compiler solves a budgeted program over the blocks of each instance; the agent receives the rendered selection.

A. TYPED BLOCK IR AND RENDERING

Each instance is converted by a deterministic adapter into typed blocks with full and, where meaningful, short variants (a tool’s name and description without its parameter schema; the first sentence of a document; the first sentence of a history session). Rendering is a pure function of the selection: blocks appear in a fixed type order (instructions, tool schemas, skill documents, examples, documents, notes, history in chronological order, and the goal last), so identical selections yield byte-identical prompts and every policy in Section VI is evaluated through the same renderer. Token costs are measured with each model’s own tokenizer.

B. PROFILING BY COUNTERFACTUAL ABLATION

For each development instance the profiler evaluates the full context and a set of ablations: every optional type removed in turn; every pair of optional types removed; each type rendered with its short variant; and a sample of single blocks removed (leave-one-block-out), which trains the instance-level model below. A length-neutral control replaces a removed type with filler text of approximately equal token length, separating the effect of content from the effect of prompt length. All runs use greedy decoding with a fixed seed and are cached by prompt hash, so repeated conditions cost nothing.

C. ESTIMATION

Type values, short-variant values and interaction terms are paired means over development instances (Definition 2), each shrunk toward zero by $n/(n + n_0)$ for n instances, which damps cells measured on few instances, and reported with paired percentile bootstrap intervals. Instance-level block values come from a ridge regression trained on

TABLE 1. A compiled plan (tool family, Qwen3-4B-Instruct profile, 1k budget): the highest-valued optional blocks, any other chosen blocks, and the two lowest-valued blocks, with predicted value $\hat{v}$ and the chosen variant.

Block	Type	Tokens	$\hat{v}$	Chosen
calculate_future_value	tool schema	162	0.447	full
calculate_compound_interest	tool schema	169	0.339	full
calculate_stock_return	tool schema	174	0.312	full
kinematics.calculate_displacement	tool schema	177	0.106	short
biological.calc_biomass	tool schema	102	0.103	full
finance.calculate_quarterly_dividend_per_share	tool schema	112	0.099	full
humidity_temperature_forecast	tool schema	107	0.080	short
ex2	example	49	0.066	full
identify_bird	tool schema	126	-0.026	–
ex0	example	52	-0.042	–

33 optional blocks; 9 chosen; 996 of 1000 tokens; status OPTIMAL, 4.8 ms

the leave-one-block-out deltas. Its features never use gold labels: block type, the type’s MCV, the fraction of the goal’s content words that occur in the block, the block’s BM25 rank within its type, log token length, for history its recency, and the cosine similarity between block and goal embeddings. A short variant’s value is the full value scaled by the type’s measured ratio $MCV_{\text{short}}(\tau)/MCV(\tau)$, clipped to $[0, 1]$.

D. THE MCV COMPILER

For instance q and budget B the compiler solves

$$\begin{aligned}
 \max \quad & \sum_{b,v} \hat{v}_{b,v} x_{b,v} + \sum_{\tau_1 < \tau_2} I(\tau_1, \tau_2) y_{\tau_1 \tau_2} \\
 \text{s.t.} \quad & \sum_v x_{b,v} \leq 1, \quad \sum_v x_{b,v} = 1 \text{ for mandatory } b, \\
 & \sum_v x_{b,v} \leq \sum_v x_{d,v} \quad \forall d \in D_b, \\
 & \sum_{b,v} c(b,v) x_{b,v} \leq B, \\
 & z_{\tau} = \max_{b: \tau_b = \tau} \sum_v x_{b,v}, \quad y_{\tau_1 \tau_2} = z_{\tau_1} z_{\tau_2},
 \end{aligned}$$

with binary variables, using the CP-SAT solver of OR-Tools under a time limit. Values are scaled to integers. The solver returns a certificate (status, objective, bound, time); if no feasible solution is found in time, a value-density greedy fills the budget and the certificate records the fallback. The additive objective never gains from a block with non-positive predicted value, so the compiler may leave budget unused: more context is not assumed to be better. With interaction terms, a block of low or negative individual value can still be selected when its type completes a positive interaction. The additive variant (OURS-A) drops the interaction terms.

Table 1 shows one compiled plan from the test split: the predicted block values, the variant chosen for each block, and the certificate. The compiler takes the full schemas of the highest-value tools, falls back to the short variant (name and description only) for tools of moderate value where the full schema would not fit, and omits blocks whose predicted value is negative.

E. TRANSFER

A profile measured on a small model is applied unchanged to a larger one (OURS-T). Fidelity is reported as the rank correlation between the two models’ type values and as the success difference between transferred and natively

measured profiles; the instance-level block-value model is shared, so the comparison isolates the type-level profile.

V. IMPLEMENTATION

The reference implementation is the open-source Python package `mcv` (1,227 lines; MIT license), with unit tests covering the edge cases the design must handle: budgets smaller than the mandatory blocks (which raise an error rather than truncating), dependency cycles and unknown dependencies, zero-value blocks, deterministic rendering order, the solver’s timeout path, and estimator recovery of known effects (18 tests, 96% line coverage of the core modules). Models run locally through Ollama with greedy decoding, a fixed seed and a fixed context window, so that the model is never reloaded between conditions; models answer in a strict text protocol (a JSON call for tools, an `ANSWER:` line otherwise) parsed by the harness. Responses are cached by a hash of (model, seed, decoding options, prompt), so a prompt produced by several policies is answered once and the reuse is logged. Token costs use each model’s Hugging Face tokenizer, and the measured token count of every rendered prompt is logged so that budget compliance can be audited. Embedding-based baselines use the open `nomic-embed-text` model [37] through the same server. The compiler uses the CP-SAT solver of OR-Tools with a time limit per instance and a single worker for determinism. Every run writes JSON-lines logs of each call, selection and solver certificate, plus a manifest with the code commit, model digests and data hashes.¹

VI. EVALUATION

A. SETUP

Task families. Each family has 300 instances, split once by a seeded shuffle into 60 development and 240 test instances, with test hashes locked before any test run. *Facts:* HotpotQA distractor questions (validation split) [38]; each instance carries its native paragraphs plus extra distractors drawn from other questions (20 documents, 2,961 tokens on average), two worked examples, and asks for a short answer. *Tool:* a BFCL v3 call from the Gorilla project’s benchmark [39] (simple and multiple categories); the correct function or functions are mixed into a 30-tool catalog (4,235 tokens), and success is an AST-level match of the emitted call. *History:* 24 synthetic chat sessions (2,718 tokens) in which one user attribute is set and later updated, a knowledge-update setting also probed by LongMemEval [40]; the agent must report the latest value, and in half of the instances a note restates the *stale* value. Success checks are deterministic.

Models. Qwen3-4B-Instruct is the main model and the profiling source; Qwen3-8B is the transfer target. Both run locally with thinking disabled, greedy decoding, a fixed seed, a 16,384-token window and at most 96 output tokens.

¹Software and data: Ollama 0.35.0; OR-Tools, scikit-learn, LLMingua and PyTorch versions pinned in `code/uv.lock`; HotpotQA (CC BY-SA 4.0) and BFCL v3 (Apache-2.0) as recorded in `data/LICENSES.md`.

Policies. B0 full context (no budget); B1 truncate oldest; B2 embedding relevance (`nomic-embed-text` cosine to the goal, fill in descending order); B3 LLMingua-2 [3] compressing all optional context to the remaining budget; B4 a PACMS-style facility-location coverage selector [2] (our re-implementation); B5 fixed type tiers; OURS-A, the MCV compiler without interaction terms; OURS, with them. All policies keep the mandatory blocks and share the renderer.

Protocol. Profiles use the first 40 development instances (4B, with 6 leave-one-block-out samples each) and 20 instances (8B, type level). Budgets are 1k, 2k, 4k and 8k tokens. The primary metric is the area under the success–budget curve (AUBC), normalized over log-budget, per instance. The main grid evaluates B0, B2, B4, OURS-A and OURS on the first 100 test instances per family; the weaker baselines B1, B3, B5 (well below B2 in dev pilots) use the first 30. Following the pre-registration, H1 compares OURS with the best of B1–B5 per family; H1b tests, at 8k, that OURS uses fewer tokens than B0 and the best baseline with success non-inferior to B0 (margin 0.05); H2 compares OURS with OURS-A pooled; H3 tests transfer. Intervals are paired percentile bootstraps (10,000 resamples); p-values are Wilcoxon signed-rank for H1 and H2 and a one-sided paired bootstrap for H1b (the plan fixed the CI rule but named no test; recorded as a deviation); H1, H1b and H2 form one Holm family. All 6,180 main episodes completed. Block costs are summed in the model’s tokenizer; rendered prompts exceeded the budget in 34 of 5,880 budgeted main-study episodes, by at most 2 tokens, because tokens merge at block boundaries.

B. MAIN RESULT: H1 IS NOT SUPPORTED

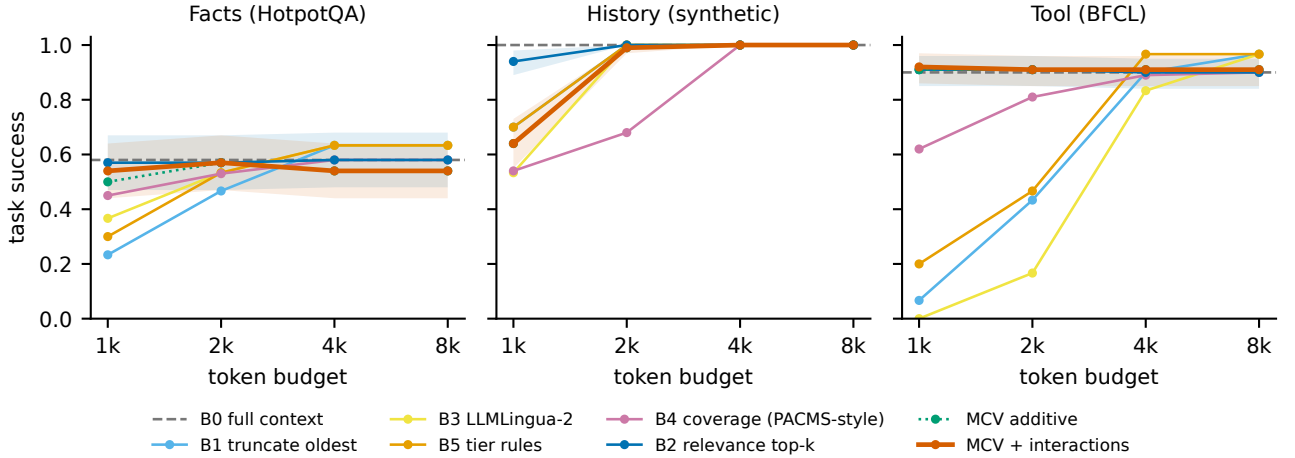
Table 3 summarizes all pre-registered tests. Table 2 and Fig. 2 give the main comparison. Embedding relevance (B2) was the best baseline in every family. OURS did not exceed it anywhere: the paired AUBC difference was -0.025 $[-0.070, 0.018]$ on facts and 0.007 $[-0.012, 0.027]$ on tool use, both indistinguishable from zero, and -0.053 $[-0.072, -0.035]$ on history, a significant *loss* (Holm $p < 0.001$). H1 required a significant gain in two families; it fails.

On tool use the MCV compiler beat the non-relevance baselines by wide margins at small budgets: at 1k it reached 92% success against 0% for LLMingua-2, 7% for truncation and 62% for coverage, because it spends the budget on the tool schemas ranked most valuable and often drops examples. This advantage did not hold elsewhere: on history at 1k OURS (64%) trailed truncation (70%) and tiers (70%). Relevance ranking also concentrates the budget on goal-related blocks. Once a block’s value is dominated by whether it is about the goal, an embedding score already captures most of what the profile adds.

Diagnosis of the history loss. Almost all of the history gap arises at the 1k budget (at 2k and above both policies are at or near ceiling). There, success depends almost

TABLE 2. Test results on Qwen3-4B-Instruct: AUBC (mean, 95% paired-bootstrap CI) and success at the 1k and 8k budgets. B1, B3 and B5 use the first 30 instances per family, the others 100. B0 ignores the budget, so its 1k and 8k entries are the same full-context run.

Policy	Facts (HotpotQA)			History (synthetic)			Tool (BFCL)		
	AUBC	@1k	@8k	AUBC	@1k	@8k	AUBC	@1k	@8k
B0 full	0.580[0.480, 0.680]	58%	58%	1.000[1.000, 1.000]	100%	100%	0.900[0.840, 0.950]	90%	90%
B1 truncate oldest	0.511[0.372, 0.650]	23%	63%	0.950[0.922, 0.978]	70%	100%	0.617[0.528, 0.700]	7%	97%
B2 relevance	0.575[0.480, 0.667]	57%	58%	0.990[0.982, 0.997]	94%	100%	0.905[0.848, 0.957]	91%	90%
B3 LLMlingua-2	0.556[0.406, 0.706]	37%	63%	0.922[0.894, 0.950]	53%	100%	0.494[0.422, 0.567]	0%	97%
B4 coverage (PACMS-style)	0.542[0.453, 0.628]	45%	58%	0.817[0.773, 0.857]	54%	100%	0.820[0.758, 0.878]	62%	90%
B5 type tiers	0.544[0.394, 0.694]	30%	63%	0.950[0.922, 0.978]	70%	100%	0.672[0.583, 0.756]	20%	97%
OURS-A (MCV additive)	0.543[0.448, 0.637]	50%	54%	0.937[0.918, 0.953]	64%	100%	0.910[0.852, 0.960]	91%	91%
OURS (MCV + interactions)	0.550[0.455, 0.645]	54%	54%	0.937[0.918, 0.953]	64%	100%	0.912[0.852, 0.962]	92%	91%

**FIGURE 2.** Success versus token budget on Qwen3-4B-Instruct (test). Bands: 95% bootstrap CIs for B2 and OURS. The dashed line is full context. MCV additive and MCV with interactions overlap almost everywhere.**TABLE 3.** Pre-registered hypotheses. Differences are paired (OURS minus comparator); H1 and H2 in AUBC, H1b and H3 in success. p : Holm-adjusted over H1, H1b and H2; H3 is decided by its CI against the margin 0.05.

Hyp.	Family	Difference [95% CI]	p	Outcome
H1	Facts	-0.025 [-0.070, 0.018]	1.000	no gain
H1	History	-0.053 [-0.072, -0.035]	<0.001	worse
H1	Tool	0.007 [-0.012, 0.027]	1.000	no gain
H1b	Facts	-0.040 [-0.100, 0.010]	1.000	not shown
H1b	History	0.000 [0.000, 0.000]	<0.001	non-inferior
H1b	Tool	0.010 [0.000, 0.030]	<0.001	non-inferior
H2	Pooled	0.003 [-0.001, 0.007]	1.000	no effect
H3	Pooled	0.000 [0.000, 0.000]	—	non-inferior

only on whether the session containing the latest update survives selection. OURS kept it in 64% of plans and B2 in 94%; without that session OURS never answered correctly (0%). The cause is the block-value model. Trained on leave-one-block-out deltas, its history coefficients weight recency (0.10) and token length (0.13) alongside embedding similarity (0.34), so it fills a small budget with the most recent sessions and misses the update whenever

it lies further back. The type-level profile was correct (history turns matter, the note adds nothing once history is present); the block-level ranking was not. The leave-one-block-out signal is sparse: removing one of 24 sessions rarely changes the outcome, so the regression learns weak correlates of value instead of value. An exploratory check on the development logs confirms this (not pre-registered; five-fold cross-validation grouped by instance). Only 5% of history-session deletions changed the outcome (5% for documents, 4% for tool schemas). The cross-validated rank correlation between predicted and observed deltas was 0.15 for the ridge model against 0.27 for embedding similarity alone on history, 0.07 against 0.17 on facts, and 0.27 for both on tool use. The learned ranker was never better than its own similarity feature.

C. EFFICIENCY: H1b IS SUPPORTED IN TWO OF THREE FAMILIES

Because the compiler adds blocks only while they raise the predicted objective, it leaves budget unused at 8k (Figs. 4 and 3), whereas B2 fills the budget and so matches full context. On tool use OURS sent 2,759 tokens instead of 4,240 (34.9% fewer) with success 91% against 90% for

full context (difference 0.010, CI [0.000, 0.030], Holm $p = <0.001$). On history it sent 6.9% fewer tokens with identical success (100%), the saving coming from the examples, the note and a few low-value sessions. On facts it sent 30.9% fewer tokens, but success fell from 58% to 54% and the CI [-0.100, 0.010] crosses the margin, so non-inferiority is not shown there. Two families pass, as H1b required. Two caveats bound this result. Full contexts average 2,961, 2,718 and 4,235 tokens, so the 8k budget never binds (nor does 4k except on tool use). H1b therefore tests pruning of full context, not selection under a binding budget. We also did not evaluate a relevance baseline with a stopping rule (for example a similarity threshold), so the savings are not shown to require MCV. They come from the block-level values, which stop adding blocks once predicted value reaches zero. On facts, full context was right and OURS wrong on 6 instances, and the reverse held on 2. Only 2 of those losses removed a supporting paragraph; in the others the model changed its answer when distractors were removed, an effect a type-level profile cannot see.

D. INTERACTIONS: H2 HAS NO DETECTABLE EFFECT

Adding interaction terms changed pooled AUBC by 0.003 [-0.001, 0.007] (Holm $p = 1.000$). History is at ceiling from 2k upward, which leaves little room for any difference, so this null result is limited. The only large interaction measured is between history and the profile note (-0.60). Because the note is current in half of the instances and stale in the other half, the negative sign means the two substitute for each other: either can supply the answer when it is current, and the note adds nothing once history is present (its own value is 0.00). The interaction terms did change the plans: OURS and OURS-A selected different blocks in 30% of facts, 50% of history and 22% of tool plans. At 8k, OURS never included the note while OURS-A included it in 49% of history plans, and the positive example-document interaction (0.10) made OURS keep an example in 100% of facts plans against 86% for OURS-A. These changes concern low-value blocks and did not change success. On these families, type-pair interactions change what is sent but not whether the task succeeds.

E. WHAT THE PROFILES SAY

Fig. 5 and Table 4 show the profiles of both models. Three regularities hold across families. First, the value of context is concentrated: tool schemas (0.85), documents (0.35) and history sessions (0.40) carry nearly all of it. Second, worked examples are worth close to nothing on their own (0.05, 0.00, -0.03 on the 4B model; the 8B model valued them at 0.10 on history), although every family’s prompt includes them by default. Third, short variants are worth much less than full blocks for tool schemas (0.30 against 0.85), because the parameter schema matters for argument filling. The length-neutral control,

TABLE 4. MCV profiles on the development split: unshrunk paired means with 95% bootstrap CIs. Rows are type values (success drop on removal), short-variant values (gain of the short variant over removal) and pairwise interactions. The 8B model was profiled at type level on fewer instances, so its intervals are wider.

Family	Term	Qwen3-4B	Qwen3-8B
Facts	base success	0.53	0.55
	example	+0.05 [0.00, +0.12]	0.00 [0.00, 0.00]
	fact	+0.35 [+0.17, +0.53]	+0.35 [+0.10, +0.60]
	short fact	+0.12 [-0.03, +0.28]	+0.20 [0.00, +0.45]
	$I(\text{example, fact})$	+0.10 [+0.03, +0.20]	0.00 [0.00, 0.00]
History	base success	1.00	1.00
	example	0.00 [0.00, 0.00]	+0.10 [0.00, +0.25]
	history turn	+0.40 [+0.25, +0.55]	+0.45 [+0.25, +0.65]
	note	0.00 [0.00, 0.00]	+0.05 [0.00, +0.15]
	short history turn	-0.03 [-0.12, +0.05]	0.00 [0.00, 0.00]
	$I(\text{example, history turn})$	0.00 [0.00, 0.00]	+0.10 [0.00, +0.25]
	$I(\text{example, note})$	0.00 [0.00, 0.00]	+0.10 [-0.10, +0.30]
	$I(\text{history turn, note})$	-0.60 [-0.75, -0.45]	-0.50 [-0.70, -0.30]
Tool	base success	0.85	0.85
	example	-0.03 [-0.07, 0.00]	-0.05 [-0.15, 0.00]
	tool schema	+0.85 [+0.72, +0.95]	+0.80 [+0.60, +0.95]
	short tool schema	+0.30 [+0.17, +0.45]	+0.25 [+0.05, +0.45]
	$I(\text{example, tool schema})$	+0.03 [-0.05, +0.10]	0.00 [-0.15, +0.15]

which replaces a removed type with filler of approximately equal token length (20 instances per type), reproduced the removal effects: documents 0.50 by removal against 0.45 with padding, tool schemas 0.90 against 0.90, history 0.45 against 0.45. The measured values reflect content, not prompt length.

F. TRANSFER ACROSS MODEL SIZE (H3)

The 4B and 8B profiles rank the 7 (family, type) values similarly (Spearman $\rho = 0.88$, pooled across families, so between-family differences contribute to the agreement), and their magnitudes agree closely (Fig. 5). On Qwen3-8B (first 40 test instances per family, budgets 2k and 8k), compiling with the transferred 4B profile and with the natively measured 8B profile gave a pooled success difference of 0.000 [0.000, 0.000] over 240 paired episodes, so H3 non-inferiority holds (Fig. 6). The outcome was the same in 100% of paired episodes, giving the degenerate interval above, although the two profiles produced identical selections in only 35%. The differences involved tool schemas, history sessions and examples alike, yet never changed the outcome. This suggests that both profiles keep the blocks that decide success and differ only in blocks that do not. It equally indicates that, with the block-value model shared, the compiler’s outcomes are insensitive to the type-level profile, so H3 does not show that the profile drives the decisions that matter. The efficiency pattern of the 4B study carried over. On tool use at 8k the transferred profile sent 2,821 prompt tokens against 4,232 for relevance ranking, with success 92% against 90%. Relevance ranking was again more accurate on facts (68% against 62% at 2k). With 40 instances per family these 8B comparisons are descriptive.

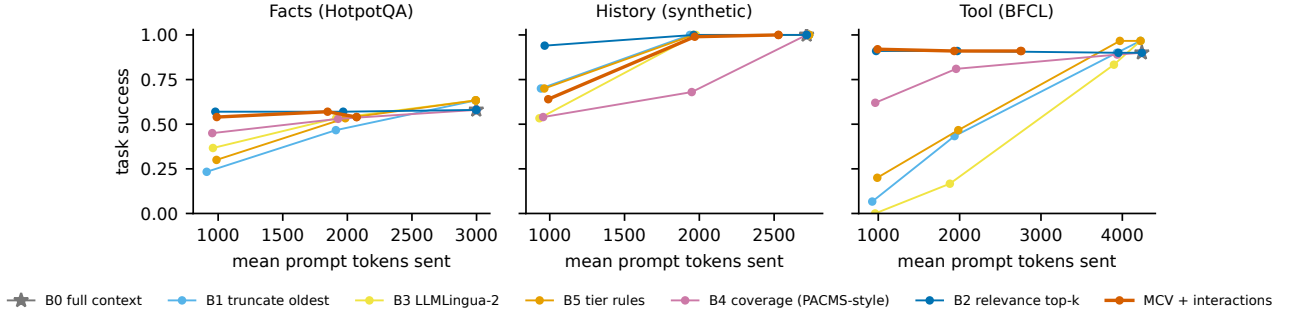


FIGURE 3. Success against the prompt tokens actually sent (test, Qwen3-4B-Instruct; one point per budget, from 1k to 8k). Points further left at equal height are more efficient. At its largest budget MCV sits left of full context (star) and of B2 on tool use and history.

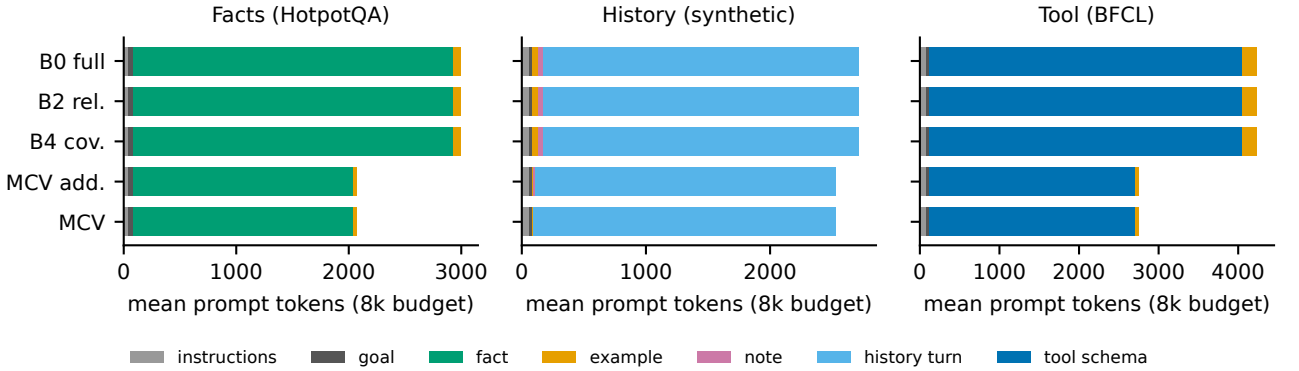


FIGURE 4. Mean prompt composition at the 8k budget (test, Qwen3-4B-Instruct). MCV leaves budget unused: it drops the profile note, most low-value documents and tool schemas, and some worked examples.

G. FAILURE ANALYSIS

We read the transcripts of failed episodes in each family; Table 5 shows one representative case per failure mode, extracted verbatim from the logs by a script. *History at small budgets:* when the compiler drops the session holding the update, the model does not guess. It reports that no conversation mentions the attribute, so these failures are abstentions rather than hallucinations. When B2 keeps the same session, the answer is correct. *Compression of identifiers:* LLMingua-2 deletes the underscores and reorders words in function and parameter names. The model then emits calls whose names do not exist, which is why B3 scores far below the selection policies on tool use at small budgets even though the call’s intent is right. Selection policies never alter block text, so they cannot fail this way. *Facts with all evidence present:* in several facts losses at 8k the supporting paragraphs were kept and only distractors were removed, yet the model’s answer changed: it answered a comparison question with a year rather than a title, or added detail that the exact-match checker rejects. Removing context changes the model’s behavior in ways a value table indexed by type cannot predict, and in these cases the change was harmful.

H. HOW MUCH PROFILING IS NEEDED?

Profiling cost scales with the number of development instances, so we asked, again as an exploratory analysis of the existing logs, how quickly the type-level profile stabilizes. We re-estimated the 4B profiles from random subsets of the 40 profiled instances (300 draws per size) and compared them with the full-sample profile (Fig. 7). With five instances per family the mean absolute error of the type values was 0.09 and the most valuable type was identified in all three families in 74% of draws. With ten instances these were 0.06 and 92%, and with twenty 0.03 and 99%. Because subsets are drawn from the same 40 instances, agreement at larger sizes is partly mechanical. The practical reading is that the type-level ranking needs on the order of ten to twenty instances per family, while the block-level model would need far more informative supervision than single-block deletions provide.

I. COST

Compilation. Over 2,400 compilations CP-SAT proved optimality in 100.0% of cases with no fallback (0); median solve time was 2.3 ms (99th percentile 22.3 ms, maximum 357 ms), and end-to-end planning including feature computation (with cached block embeddings) took

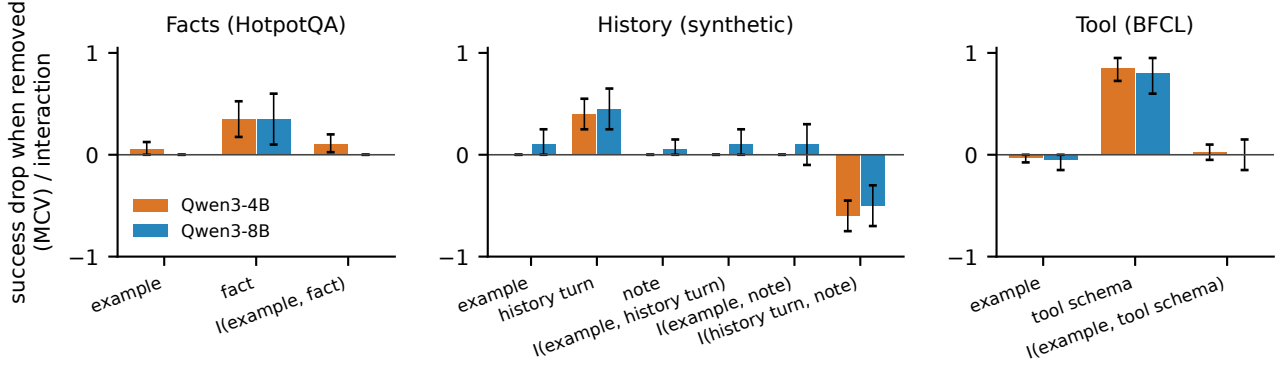


FIGURE 5. MCV profiles (development split): success drop when a type is removed, and pairwise interactions $I(\tau_1, \tau_2)$, for Qwen3-4B-Instruct (40 instances) and Qwen3-8B (20 instances), with 95% bootstrap CIs.

TABLE 5. Representative failures (test split, Qwen3-4B-Instruct), extracted verbatim from the logs. Long texts are truncated.

Case	Goal and context	Replies (verbatim)
History, 1k	Question: What is the user's current car? Session holding the update: "User: "By the way, I drive a white Tesla Model 3 now.""'. Kept by B2: yes; by MCV: no.	MCV: ANSWER: None of the provided conversations mention the user's current car. B2: ANSWER: white Tesla Model 3
Tool, 1k	Request: Calculate the future value of my investment of \$1000 with an annual interest rate of 5% over 2 years.	MCV: {"name": "calculate_future_value", "arguments": {"initial_investment": 1000, "interest_rate": 0.05, "duration": 2}} B3: {"name": "calculate future value", "arguments": {"investment initial": "1000", "yearly interest": "5", "compounded": "1", "holding period": "2"}} MCV: ANSWER: 1963 B0: ANSWER: Miracle of the White Stallions
Facts, 8k	Question: Which film was released first out of The Hunchback of Notre Dame and Miracle of the White Stallions?	

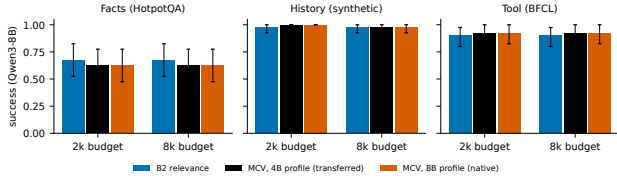


FIGURE 6. Transfer to Qwen3-8B (test, 40 instances per family): relevance ranking, MCV with the transferred 4B profile, and MCV with the native 8B profile, with 95% CIs.

a median 6.9 ms. **Profiling.** The 4B profile cost 1,440 model calls (3.0 hours on a laptop). Measured in prompt tokens, profiling pays for itself after about 931 queries on tool use and 1,016 on facts, given the 8k-budget savings above. The facts figure is conditional on accepting that family's non-significant success loss. History saves less per query and breaks even after about 5,901 (Fig. 8).

J. DEVELOPMENT HISTORY

The block-value model went through three versions on development data only. The first used lexical features (dev AUBC 0.633); the second added embedding similarity (0.758); the third, which apportioned type value across blocks, scored 0.736 and was rejected by the rule fixed

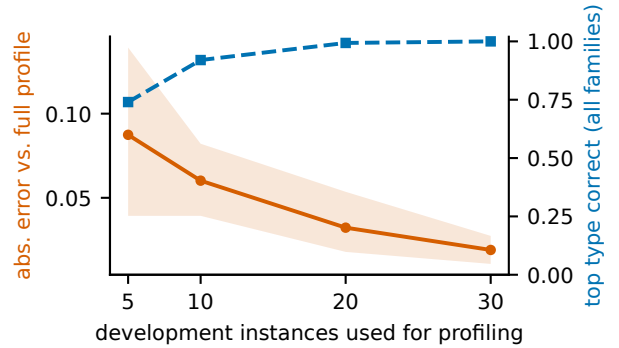


FIGURE 7. Profile stability versus development instances (exploratory; subsamples of the 4B profiling logs). Band: the range of the middle four fifths of draws.

before it ran. The second version was frozen and pre-registered before any test run. All three pilots, and a crashed pilot run, are in the released logs.

VII. DISCUSSION, LIMITATIONS AND THREATS TO VALIDITY

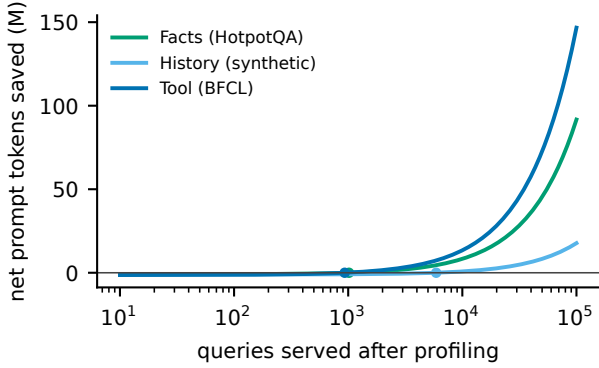


FIGURE 8. Net prompt tokens saved at the 8k budget after paying for 4B profiling, as a function of queries served. Dots mark break-even.

A. WHAT THE RESULTS MEAN

The evidence separates two uses of measured context value. As a *selector* the MCV compiler was no better than embedding relevance, and worse when its block-level ranking learned the wrong correlate. As a *diagnostic* it was useful. It showed that worked examples had near-zero individual value on the 4B model, that the profile note adds nothing once history is present, and that tool schemas lose most of their value when shortened. Separately, the block-level values, which stop at zero, cut prompt tokens substantially on tool use without losing success; a relevance ranker with a stopping rule, which we did not test, might do the same. The compiler also stopped spending once the remaining blocks added no predicted value, which a fill-the-budget selector does not do. These findings suggest a hybrid in which relevance ranks blocks within a type and the profile decides which types to include and when to stop. We did not test that hybrid, and we state it only as the design the data point to.

The negative result is informative about leave-one-block-out supervision. When many blocks are individually dispensable (one session of 24, one document of 20), single deletions rarely change the outcome. A regression on those deltas then learns proxies such as recency and length. Group or pairwise deletions, or using relevance directly as the within-type ranker, are natural fixes.

B. PRACTICAL GUIDANCE

For practitioners the results suggest a cheap procedure. Profile each (model, task family) on a small development set (ten to twenty instances sufficed here to rank types, judged from subsamples of the same 40 instances) using type removals and the length-neutral control. Consider removing types whose value interval sits at zero (here, worked examples on history and tool use) from the default prompt, after checking their interactions. Keep a strong relevance ranker for choosing blocks within a type. Stop adding context when the remaining types have no measured value rather than filling the budget. Re-profile

when the model or the task family changes; the 4B-to-8B agreement observed here is encouraging but limited to one model family. Interaction terms can be omitted unless the profile shows a large interaction between two types that both carry value.

C. LIMITATIONS AND THREATS TO VALIDITY

Construct validity. Each instance is a single agent decision, not a multi-step trajectory with a simulated user as in τ -bench [41]; context value in long-horizon agents may compound across steps. Success is a deterministic check (answer match, AST-level call match), which misses partially correct outputs. **Internal validity.** B4 is our re-implementation of a PACMS-style objective, not the authors' code, and B3 uses one LLMingua-2 checkpoint; both could be tuned further. Rendered prompts may exceed the budget by at most 2 tokens because tokenization is not additive across block boundaries. Reading transcripts after the test runs revealed a defect in the tool checker: it compared list-valued arguments as strings, so an integer list failed against the same values written as floats. Rescoring all logged answers with element-wise numeric comparison changed 22 episodes, spread across all policies, and left the H1, H1b and H2 mean paired differences unchanged at three decimals (for example H1 on tool use, 0.007). We keep the pre-registered checker as primary. Greedy decoding on Apple-silicon inference is not guaranteed to be bitwise deterministic; caching makes every reported response reproducible from the logs, but a fresh rerun may differ slightly. **Statistical conclusion validity.** The pre-registered power analysis put power for a 0.05 AUBC effect on facts at about 58% with 100 instances, so the null result on facts is weak evidence of no effect. The weaker baselines used 30 instances; each scored below B2 in every family (Table 2), so B2 is the best baseline throughout. Transfer used 40 instances per family and two budgets. **External validity.** Two models of one family (Qwen3, 4B and 8B) were tested; a third model from another family was planned and dropped for compute, so transfer across model families is untested. One task family (history) is synthetic; HotpotQA is known to admit reasoning shortcuts that MuSiQue was built to remove [42]; and HotpotQA and BFCL may overlap with pretraining data. Profiles are per (model, task family) and must be re-measured when either changes, at the profiling cost reported in Section VI.

VIII. CONCLUSION

Marginal context value profiles measure, per model and task family, how much each kind of context changes success. A budget compiler can then act on them with certificates and millisecond latency. In a pre-registered local evaluation the compiled context did not beat embedding relevance in area under the success-budget curve, and it lost on history because its block-level ranker learned recency instead of relevance. It did reduce prompt tokens at a non-binding budget (a hypothesis added before testing)

by 34.9% on tool use and 6.9% on history without losing success, and its profiles exposed which context types were worth sending at all. We release the method, data adapters, logs and analysis so that the profiles, and the negative result, can be checked and extended.

Use of AI assistance. An AI coding assistant was used to write and test code, run the experiments, and draft and edit text. All numbers in the paper are generated from the released logs by scripts, and every reference was verified against its DOI record. The author reviewed the content and is responsible for it.

Artifact availability. The `mcv` package, data adapters (with the licenses of HotpotQA and BFCL), pre-registration, raw logs of every model call, and the scripts that regenerate all figures, tables and numbers with one command will be released under an open license with the preprint.

References

- [1] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, “A Survey of Context Engineering for Large Language Models,” arXiv preprint arXiv:2507.13334, 2025.
- [2] M. Ghulyani, A. Singh, K. Bharadwaj, A. Nath, and S. Goswami, “PACMS: Submodular Context Selection as a Pluggable Engine for LLM Agents,” arXiv preprint arXiv:2606.20047, 2026.
- [3] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression,” arXiv preprint arXiv:2403.12968, 2024.
- [4] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, 2024.
- [5] B. Cohen-Wang, H. Shah, K. Georgiev, and A. Madry, “ContextCite: Attributing Model Generation to Context,” arXiv preprint arXiv:2409.00729, 2024.
- [6] F. Liu, N. Kandpal, and C. Raffel, “AttriBoT: A Bag of Tricks for Efficiently Approximating Leave-One-Out Context Attribution,” arXiv preprint arXiv:2411.15102, 2024.
- [7] S. Xie, M. Luo, C. D. Stern, M. Du, and L. Cheng, “DemoShapley: Valuation of Demonstrations for In-Context Learning,” arXiv preprint arXiv:2410.07523, 2024.
- [8] H. Liu, X. Mao, H. Xia, J. Lou, J. Liu, and K. Ren, “Prompt Valuation Based on Shapley Values,” arXiv preprint arXiv:2312.15395, 2023.
- [9] A. Ghorbani and J. Zou, “Data Shapley: Equitable Valuation of Data for Machine Learning,” arXiv preprint arXiv:1904.02868, 2019.
- [10] T. Li, J. Liu, Q. Zhao, Y. Li, L. Wang, B. Shao, X. Liu, and L. Shou, “What Is a Skill Worth? Structure-Aware Shapley Valuation of Agent Skills,” arXiv preprint arXiv:2608.04562, 2026.
- [11] P. Liu, X. Wang, D. Liang, and W. Ye, “The Laws of Context Allocation: Causal Measurement and Closed-Loop Orchestration in Generative Search,” arXiv preprint arXiv:2608.23252, 2026.
- [12] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou, and K. Olukotun, “Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models,” arXiv preprint arXiv:2510.04618, 2025.
- [13] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,” arXiv preprint arXiv:2310.03714, 2023.
- [14] P. Xu, Z. Zhang, Y. Sun, F. Tian, L. Wang, and C. Zhang, “ContextPipe: Database-Inspired Context Assembly for Long-Horizon Agents,” arXiv preprint arXiv:2609.00749, 2026.
- [15] S. Kashmira, J. L. Dantanarayana, L. Tang, and J. Mars, “ContextRender: From Execution Dependencies to Agent Context,” arXiv preprint arXiv:2609.37743, 2026.
- [16] D. Paul, “Context Assembly as the Controlled Variable: A Control-Theoretic View of Harness Policies for Frozen LLM Agents,” arXiv preprint arXiv:2607.25408, 2026.
- [17] W. Li, J. Xu, A. Chen, X. Wang, S. Liang, J. Liang, and D. Yang, “Beyond Skill Evolution: Self-Evolving Context Management Policies for Long-Horizon Agent Harnesses,” arXiv preprint arXiv:2609.34649, 2026.
- [18] Y. Lin, E. Ang, E. Zhu, B. Ding, and J. Zhou, “Context as an Environment: Programmatic Context Management for Long-Horizon Agents,” arXiv preprint arXiv:2608.21690, 2026.
- [19] C. Wu, S. Yang, and A. Murali, “Imprompt: A Language Framework for Prompt Programming,” arXiv preprint arXiv:2607.22683, 2026.
- [20] Y. Chen, R. Chen, X. Wang, Z. Li, and L. Huang, “Optimal Skill Selection for LLM Agents with Provable Bicriteria Guarantees,” arXiv preprint arXiv:2608.19993, 2026.
- [21] Z. Dong, F. Qin, M. Shah, and Y. Wang, “Know Before You Fetch: Calibrated Retrieval-Budget Allocation for Retrieval-Augmented Generation,” arXiv preprint arXiv:2606.29959, 2026.
- [22] A. N. Bala, “Recall Is Not Enough: A Reader-Context Diagnostic for Budget-Constrained Retrieval-Augmented Generation,” arXiv preprint arXiv:2607.00725, 2026.
- [23] Y. Wu, Y. Zhang, S. Ghosh, S. Basu, A. Deoras, J. Huan, and G. Gupta, “ContextWeaver: Selective and Dependency-Structured Memory Construction for LLM Agents,” arXiv preprint arXiv:2604.23069, 2026.
- [24] K. Gwimm and C. Eisenach, “Beyond the Harness: End-to-End Optimization of Context Artifacts for Enterprise Text-to-SQL,” arXiv preprint arXiv:2608.22830, 2026.
- [25] F. Sakizli, “Tool-Schema Compression Enables Agentic RAG Under Constrained Context Budgets,” arXiv preprint arXiv:2605.26165, 2026.
- [26] A. K. G. Rao and A. Jaggi, “BudgetBench: A Budget-Tiered Protocol and Pilot Harness for Memory Strategy Evaluation in Local Large Language Model Agents,” arXiv preprint arXiv:2609.13149, 2026.
- [27] J. Tang, S. Liu, Z. Zhang, Y. Yuan, L. Zheng, W. Su, and B. Zheng, “COMI: Coarse-to-fine Context Compression via Marginal Information Gain,” arXiv preprint arXiv:2602.01719, 2026.
- [28] W. Johnson and C. Lee, “Prompt Compression in Production Task Orchestration: A Pre-Registered Randomized Trial,” arXiv preprint arXiv:2603.23525, 2026.
- [29] Y. Hou and Z. Yang, “Control Under Compression: Reliability Frontiers for Tool-Using Agents,” arXiv preprint arXiv:2608.01056, 2026.
- [30] J. Deng, Y. Shen, Z. Pei, Y. Chen, and L. Huang, “Influence Guided Context Selection for Effective Retrieval-Augmented Generation,” arXiv preprint arXiv:2509.21359, 2025.
- [31] X. Guan, Q. Zhao, and Y. Deng, “Decision-Aware Memory Cards: Counterfactual-Inspired Context Selection and Compression for Tool-Using LLM Agents,” arXiv preprint arXiv:2606.08151, 2026.
- [32] M. Wang, B. Wang, F. Luo, G. Yao, C. Ning, Y. Guo, H. Chen, Y. Ma, and J. Han, “DRSR: Learning Set-Level Deletion Risk for Efficient Long-Horizon Agents,” arXiv preprint arXiv:2609.27276, 2026.
- [33] B. Yang, J. Sun, Z. Lu, Z. Qiu, X. Peng, and J. Zheng, “McClear: Cooperative Game-Theoretic Attribution and Risk-Aware Memory Clearance for Long-Horizon LLM Agents,” arXiv preprint arXiv:2609.09115, 2026.
- [34] Y. Li, W. Liu, B. Gao, J. Xie, Z. Ai, N. Zou, Y. Li, and T. Fu, “SkillsInjector: Dynamic Skill Context Construction for LLM Agents,” arXiv preprint arXiv:2605.29794, 2026.
- [35] Z. Zhang, M. Duan, and H. R. Zhang, “Efficient Estimation of Kernel Surrogate Models for Task Attribution,” arXiv preprint arXiv:2602.03783, 2026.

- [36] P. Khatri, “Do Context Files Help Coding Agents? A Two-Agent Ablation Study on Real Repositories,” arXiv preprint arXiv:2607.27250, 2026.
- [37] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic Embed: Training a Reproducible Long Context Text Embedder,” arXiv preprint arXiv:2402.01613, 2024.
- [38] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering,” arXiv preprint arXiv:1809.09600, 2018.
- [39] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large Language Model Connected with Massive APIs,” arXiv preprint arXiv:2305.15334, 2023.
- [40] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, “LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory,” arXiv preprint arXiv:2410.10813, 2024.
- [41] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “\$\$-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains,” arXiv preprint arXiv:2406.12045, 2024.
- [42] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “MuSiQue: Multihop Questions via Single-hop Question Composition,” *Transactions of the Association for Computational Linguistics*, 2022.

VENKATA SANGARAJU (Member, IEEE) is currently an Independent Researcher working on reliability and governance for enterprise artificial intelligence (AI) agents.